



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Ing. En Sistemas Computacionales

Tópicos de IA

10:00 – 11:00

Bonus 1

Alumnas:

Gavilanes Medina Tania Elizabeth

Pacheco Reyes Kimberlyn

Ramirez Pompa Marian Gabriela

Maestro: Zuriel Dathan Mora Felix

INDICE

Problema de programación de trabajo (JSSP)	3
Problema de las N reynas.....	6
Árbol de expansión mínima (MST).....	10
Problema del agente viajero (TSP).....	13

Problema de programación de trabajo (JSSP)

Definición

El **Job-Shop Scheduling Problem (JSSP)** es un problema de optimización combinatoria que se centra en la asignación de recursos compartidos a lo largo del tiempo para actividades que compiten entre sí. Este problema es fundamental en la industria de operaciones y ha sido objeto de estudio durante más de seis décadas.

En un problema JSSP se tienen:

- **n trabajos** ($J_1, J_2, \dots, J_n$) que deben ser procesados
- **m máquinas** ($M_1, M_2, \dots, M_m$) como recursos compartidos
- Cada trabajo consiste en una secuencia de **operaciones** ($O_1, O_2, \dots, O_k$) que deben ejecutarse en un orden específico
- Cada operación requiere una máquina particular y tiene un **tiempo de procesamiento** definido (p)

El objetivo principal es determinar la secuencia óptima de operaciones en cada máquina que **minimice el makespan** (C_{max}), es decir, el tiempo total de finalización de todos los trabajos.

Suposiciones Fundamentales

Para la formulación clásica del JSSP se establecen las siguientes suposiciones :

1. **Cada trabajo tiene un número finito de operaciones**
2. **El tiempo de procesamiento de cada operación está definido** y es conocido
3. **Existe una secuencia predefinida** de operaciones que debe mantenerse para completar cada trabajo
4. **Una máquina solo puede procesar un trabajo a la vez**
5. **Cada trabajo visita cada máquina solo una vez**
6. **El sistema no puede ser interrumpido** hasta que cada operación finalice
7. **No hay costos de preparación** ni costos por tardanza en la formulación básica.

Complejidad Computacional

El JSSP es un problema **NP-Hard** (NP-difícil) . Esto tiene implicaciones fundamentales:

- No se conoce un algoritmo que pueda resolver todas las instancias del problema en tiempo polinomial
- El tiempo de ejecución para algoritmos exactos crece exponencialmente con el tamaño del problema
- Para instancias de tamaño considerable (más de 20 trabajos con 20 máquinas), la búsqueda exhaustiva es computacionalmente inviable

Para el caso específico de **m > 2 máquinas**, el problema es **NP-completo**

Representación Matemática de JSSP

En la notación matemática, estos son algunos de los elementos fundamentales:

SÍMBOLO	SIGNIFICADO	EJEMPLO
J	Conjunto de trabajos	$J = \{J_1, J_2, J_3\} \rightarrow 3$ trabajos
M	Conjunto de máquinas	$M = \{M_1, M_2\} \rightarrow 2$ máquinas
N	Número de trabajos	$n = 3$
M	Número de máquinas	$m = 2$
O_{ij}	Operación j del trabajo i	$O_{23} \rightarrow$ operación 3 del trabajo 2
P_{ij}	Tiempo que dura la operación O _{ij}	$p_{23} = 5 \rightarrow$ dura 5 unidades de tiempo

Representación con Matriz de Operaciones

La forma más común de representar un problema JSSP es mediante una **matriz de tiempos de procesamiento** y una **matriz de secuencia de máquinas**.

Ejemplo Práctico

Suponiendo que se tienen 3 **trabajos** y 3 **máquinas**:

Matriz de Tiempos de Procesamiento (P)

Cada celda indica cuánto tiempo dura esa operación:

TRABAJO	OP 1 (M_1)	OP 2 (M_2)	OP 3 (M_3)
J_1	3	2	4
J_2	5	1	3
J_3	2	4	6

Interpretación: El trabajo J_1 primero pasa por M_1 y tarda 3 unidades, luego por M_2 y tarda 2, finalmente por M_3 y tarda 4.

Matriz de Secuencia de Máquinas (Q)

Indica en qué orden visita las máquinas cada trabajo:

TRABAJO	OP 1	OP 2	OP 3
J_1	M_1	M_2	M_3
J_2	M_1	M_3	M_2
J_3	M_2	M_1	M_3

Interpretación: J_2 comienza en M_1 , luego va a M_3 , y termina en M_2 .

Variables de Decisión

Para modelar matemáticamente el problema, se necesita definir **qué se esta decidiendo**. La variable principal es:

S_{ij} = Tiempo de inicio de la operación j del trabajo i

Por ejemplo:

- $S_{11} = 0 \rightarrow$ La primera operación del trabajo 1 comienza en tiempo 0
- $S_{22} = 7 \rightarrow$ La segunda operación del trabajo 2 comienza en tiempo 7

Conociendo S_{ij} y p_{ij} , se puede calcular el tiempo **de finalización**:
 $C_{ij} = S_{ij} + p_{ij}$

Problema de las N reynas

Definición y Conceptos Fundamentales

El **Problema de las N Reinas** es un problema de colocación de piezas en un tablero de ajedrez. Su enunciado es aparentemente simple, pero esconde una complejidad combinatoria fascinante:

Colocar N reinas en un tablero de ajedrez de $N \times N$ de tal manera que ninguna reina ataque a otra.

Las reinas en ajedrez se mueven horizontalmente, verticalmente y en diagonal (en todas direcciones y sin límite de distancia). Por lo tanto, la restricción implica que:

- No puede haber dos reinas en la misma fila
- No puede haber dos reinas en la misma columna
- No puede haber dos reinas en la misma diagonal (tanto ascendente como descendente)

Historia y Origen

El problema fue propuesto originalmente por el ajedrecista alemán Max Bezzel en 1848 para el caso de 8 reinas (el problema clásico de las 8 reinas). Posteriormente, Franz Nauck publicó las primeras soluciones en 1850 y extendió el problema a N reinas. Carl Friedrich Gauss también trabajó en este problema, aunque inicialmente creyó haber encontrado 72 soluciones cuando en realidad son 92.

Formalización Matemática

Representación del Problema

Sea un tablero de tamaño $N \times N$. Podemos representar una configuración de reinas como un vector:

$$S = (q_1, q_2, \dots, q_N)$$

donde q_i representa la columna en la que se coloca la reina en la fila (asumiendo una reina por fila, lo que automáticamente satisface la restricción de filas únicas).

Restricciones Matemáticas

Con esta representación, las restricciones se expresan como:

1. Restricción de columnas únicas:

$$q_i \neq q_j \forall i \neq j$$

(No dos reinas en la misma columna)

2. **Restricción de diagonales:**

$$|q_i - q_j| \neq |i - j| \quad \forall i \neq j$$

(La diferencia de columnas no debe ser igual a la diferencia de filas)

Número de Soluciones

N	Número de Soluciones Distintas	Soluciones Únicas (considerando simetrías)
1	1	1
2	0	0
3	0	0
4	2	1
5	10	2
6	4	1
7	40	6
8	92	12
9	352	46
10	724	92
11	2,680	341
12	14,200	1,787
13	73,712	9,233
14	365,596	45,752
15	2,279,184	285,053
16	14,772,512	1,846,955
17	95,815,104	11,977,939
18	666,090,624	83,263,591

Complejidad Computacional

El Problema de las N Reinas es un problema **NP-Completo** en su formulación de decisión y es un problema **P-Completo** para contar el número de soluciones.

- **Espacio de búsqueda:** N^N configuraciones posibles si consideramos todas las asignaciones
- **Espacio reducido:** Configuraciones considerando una reina por fila y columna
- **Complejidad temporal:** La búsqueda exhaustiva es lo que crece factorialmente

Para $N=20$, hay aproximadamente 2.4×10^{18} configuraciones posibles, lo que hace inviable la enumeración completa.

Aplicaciones y Generalizaciones

El problema de las N Reinas tiene aplicaciones más allá del ajedrez:

Aplicaciones Prácticas

- **Diseño de VLSI:** Colocación de componentes en chips
- **Asignación de recursos:** Problemas de scheduling y asignación
- **Memoria paralela:** Diseño de sistemas de memoria sin conflictos
- **Criptografía:** Generación de permutaciones seguras
- **Algoritmos de búsqueda:** Benchmark para probar técnicas de optimización

Variantes del Problema

- **N-Reinas con obstáculos:** Tablero con celdas bloqueadas
- **N-Reinas torre:** Usando torres en lugar de reinas
- **Problema de las reinas dominantes:** Minimizar reinas que dominan todo el tablero
- **N-Reinas con colores:** Múltiples conjuntos de reinas de diferentes colores

Representación

En lugar de representar un tablero completo de $N \times N$ celdas, es mejor representar la solución como una lista simple de N elementos, donde:

- La posición en la lista (índice) representa la fila del tablero
- El valor almacenado en esa posición representa la columna donde está colocada la reina

Ejemplo de estado parcial para 4 reinas:

[2, 5, 3, -1]

Esto significa:

- Fila 0: reina en columna 2
- Fila 1: reina en columna 5
- Fila 2: reina en columna 3
- Filas 3: aún sin reina

Referencias

<https://zbmath.org/0782.90054>

https://en.wikipedia.org/wiki/Eight_queens_puzzle

<https://www.cs.utexas.edu/~mitra/csSpring2022/cs313/assgn/assgn13.html>

<https://liacs.leidenuniv.nl/~kosterswa/nqueens/index.html>

<https://dl.acm.org/doi/10.1145/131214.131227>

<https://dl.acm.org/doi/10.1016/0012-365X%252875%252990079-5>

Árbol de expansión mínima (MST)

Definición

Un **Árbol de Expansión Mínima (MST)** de un grafo no dirigido, conexo y ponderado es un subconjunto de aristas que cumple dos condiciones fundamentales:

1. **Conecta todos los vértices** del grafo original (es un *spanning tree*).
2. La **suma total de los pesos de sus aristas es la mínima posible** entre todos los árboles de expansión del grafo.

Historia y Origen

El problema del MST es anterior al nacimiento de la propia ciencia de la computación. Sus orígenes se remontan a **1926**, cuando el matemático checo **Otakar Borůvka** publicó la primera solución polinomial, motivada por el diseño de una red eléctrica eficiente en Moravia.

Hitos históricos clave:

- **1926:** Algoritmo de Borůvka .
- **1930:** Algoritmo de Jarník (redescubierto posteriormente por Prim) .
- **1956:** Algoritmo de Kruskal.
- **1957:** Algoritmo de Prim .
- **1987:** Uso de Fibonacci Heaps para optimización (Fredman & Tarjan)

Algoritmos Principales

Existen dos algoritmos voraces (*greedy*) clásicos para resolver este problema:

Algoritmo de Kruskal

Se centra en las **aristas**.

1. Ordena todas las aristas del grafo de menor a mayor peso.
2. Añade la arista de menor peso al MST, siempre y cuando no forme un ciclo con las aristas ya seleccionadas.
3. Repite hasta que haya $n - 1$ aristas.

Es ideal para grafos dispersos (con pocas aristas).

Algoritmo de Prim

Se centra en los **vértices**.

1. Comienza con un vértice arbitrario y lo marca como visitado.
2. En cada paso, busca la arista de menor peso que conecte un vértice visitado con uno no visitado.
3. Añade esa arista y el nuevo vértice al conjunto visitado.
4. Repite hasta incluir todos los vértices.

Es muy eficiente para grafos densos (con muchas aristas).

Aplicaciones en el Mundo Real

El problema del MST no es solo un ejercicio académico; tiene aplicaciones directas en la optimización de redes y sistemas:

- **Redes de Telecomunicaciones:** Diseñar la red de cableado más económica para conectar oficinas.
- **Redes Eléctricas:** Construcción de redes de distribución eficientes.
- **Transporte:** Planeación de carreteras o vías férreas.
- **Clustering y Clasificación:** Agrupación de datos en Machine Learning.
- **Biología Computacional:** Construcción de árboles filogenéticos.

Representación

Criterio	Lista de Adyacencia	Matriz	Lista de Aristas
Memoria	Eficiente (proporcional a aristas)	Ineficiente (V^2)	Muy eficiente
Algoritmo recomendado	Prim (con Priority Queue)	Prim (versión simple)	Kruskal

Criterio	Lista de Adyacencia	Matriz	Lista de Aristas
Facilidad de implementar	Media	Alta	Muy Alta
Verificar si A y B están unidos	Lento (hay que recorrer la lista)	Instantáneo	Muy lento

Si se va a usar **Kruskal**, mejor una **Lista de Aristas**. Porque Kruskal no camina por el grafo, sino que selecciona cables sueltos y los une.

Y si se va a usar **Prim**, mejor una **Lista de Adyacencia**. Porque permite explorar los vecinos del nodo actual de forma muy rápida.

Referencias

<https://www.cambridge.org/core/books/abs/discrepancy-method/minimum-spanning-trees/9680B7DC331FB2D0C324F6B519A55D56>

<https://www.sciencedirect.com/topics/mathematics/minimum-spanning-tree>

<https://mathworld.wolfram.com/MinimumSpanningTree.html>

https://uchile.progcomp.cl/apunte/grafos/minimum_spanning_tree/#:~:text=El%20algoritmo%20de%20Kruskal%20ordena%20las%20aristas,dos%20nodos%20son%20parte%20del%20mismo%20conjunto.

<https://www.studysmarter.es/resumenes/matematicas/matematicas-de-la-decision/el-metodo-del-arbol-de-expansion-minimo/>

Problema del agente viajero (TSP)

Definición Formal

El **Problema del Agente Viajero (TSP)** pregunta lo siguiente: *"Dada una lista de ciudades y las distancias entre cada par de ciudades, ¿cuál es la ruta más corta posible que visite cada ciudad exactamente una vez y regrese a la ciudad de origen?"*.

En términos de teoría de grafos:

- Las **ciudades** son los **vértices** de un grafo.
- Las **conexiones** entre ciudades son las **aristas**.
- Las **distancias** son los **pesos** de las aristas.

El objetivo es encontrar un ciclo que visita cada vértice exactamente una vez de peso mínimo.

Complejidad Computacional

El TSP es un problema **NP-duro** (NP-hard) en optimización combinatoria. Esto tiene implicaciones fundamentales:

- **Versión de decisión:** Preguntar "¿existe una ruta de costo menor o igual a L ?" es un problema **NP-completo**.
- **Implicación práctica:** Es poco probable que exista un algoritmo que resuelva todas las instancias del TSP en tiempo polinomial. El tiempo de ejecución en el peor caso para algoritmos exactos crece de forma superpolinomial (típicamente exponencial) con el número de ciudades

Métodos Aproximados (heurísticos y metaheurísticos)

Método	Tipo	Descripción
Vecino Más Cercano	Constructivo	En cada paso, ir a la ciudad más cercana no visitada. No garantiza optimalidad.
Algoritmo de Christofides-Serdyukov	Aproximación	Garantiza una solución a lo sumo 1.5 veces la óptima para TSP métrico.
Búsqueda Local (2-opt, 3-opt)	Mejoramiento	Intercambia aristas para eliminar cruces en la ruta.
Algoritmos Genéticos	Metaheurística	Simula evolución poblacional de rutas.
Recocido Simulado	Metaheurística	Permite movimientos empeorantes para escapar de óptimos locales

Formulación Matemática (Programación Lineal Entera)

Una de las formulaciones más conocidas es la de **Dantzig-Fulkerson-Johnson** :

Variables: $x_{ij} \in \{0,1\}$ indica si la arista (i,j) está en la ruta.

Función objetivo (minimizar costo total):

$$\min \sum_{i=0}^n \sum_{j \neq i}^n c_{ij} x_{ij}$$

Restricciones de grado (cada ciudad tiene grado 2 en el ciclo):

$$\sum_{i=0, i \neq j}^n x_{ij} = 1 \forall j$$
$$\sum_{j=0, j \neq i}^n x_{ij} = 1 \forall i$$

Restricciones de eliminación de subtours (evitan ciclos desconectados):

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1 \forall S \subset V, S \neq \emptyset, S \neq V$$

Aplicaciones en el Mundo Real

El TSP tiene aplicaciones en numerosos campos:

Área	Aplicación
Logística y Transporte	Optimización de rutas de reparto, recolección de residuos, rutas escolares.
Manufactura	Perforación de placas de circuito impreso (PCB), minimización de cambios de herramienta.
Biología Computacional	Secuenciación de ADN, construcción de árboles filogenéticos.

Astronomía	Minimización del tiempo de movimiento del telescopio entre observaciones.
Robótica	Planificación de rutas para robots de soldadura o mecanizado.
Almacenes	Optimización de rutas de picking (recolección de productos)

Representación

Para este problema es mejor usar una matriz porque el TSP trabaja con grafos completos, por lo que no hay ahorro significativo usando listas de adyacencia, pero sí pierdes el acceso constante $O(1)$ que es crucial.

Referencias

https://en.wikipedia.org/wiki/Travelling_salesman_problem

https://coral.ise.lehigh.edu/wiki/doku.php/travelling_salesman_problem

<https://www.cambridge.org/core/books/approximation-algorithms-for-traveling-salesman-problems/bibliography/C84AE746BAFB48D127B0952B0337C91A>

https://glossary.cs.uwlax.edu/mw/index.php?title=Traveling_salesman_problem

https://pure.mpg.de/pubman/faces/ViewItemFullPage.jsp?itemId=item_3636255